

Tunify:

Instruments, Accessories & Parts E-Commerce System

Name:

Benitez, Alwin C.
Roduta, Kelly Laurence Q.

Section:

BSIT – S – 2A

Submitted to:

Mr. Dalisay

May 2026

FUNCTIONAL REQUIREMENTS

The Tunify system uses Node.js Express backend, MySQL2 database, and jQuery frontend. Authentication is implemented with JWT and bcrypt. Image uploads are handled by Multer, email notifications via Nodemailer, and UI uses Bootstrap. The frontend includes jQuery DataTables for data management with PDF/Excel export, Chart.js for dashboard charts, SweetAlert2 for alerts, and Bootbox for confirmations. Cart data is persisted using localStorage and session tokens via sessionStorage. The system implements full CRUD operations across user accounts, musical instruments and accessories, shopping cart, and orders, all protected by JWT middleware with role-based access control.

1. Functional Requirements

1.1 User Account Management

This module handles user registration, secure login using JWT (JSON Web Token) authentication, and account maintenance. Built with Node.js, Express, bcrypt for password hashing, and MySQL2 as the database driver, it ensures users can safely access the system and manage their profiles via vanilla JavaScript AJAX calls on the frontend.

ID	Requirement	Description
FR1.1.1	User Registration	The system shall allow users to register an account by submitting name, email, and password via an AJAX POST request; the password is hashed using bcrypt before being stored in the MySQL users table.
FR1.1.2	Login Validation	The system shall validate login credentials, compare the submitted password against the bcrypt-hashed value stored in the database, and return a signed JWT token upon success for subsequent authenticated requests.
FR1.1.3	Profile Update	The system shall allow users to update personal information such as first name, last name, address line, zip code, phone number, and profile image by sending a multipart/form-data POST request; image uploads are handled by Multer and stored in the server images directory.
FR1.1.4	Account Deactivation	The system shall allow users to soft-delete their account by setting the deleted_at timestamp in the users table via a DELETE request; deactivated accounts are excluded from login queries.
FR1.1.5	Session Management	The system shall store the JWT token and user ID in sessionStorage on the client side upon successful login, and clear them upon logout, redirecting the user to the login page.

ID	Requirement	Description
FR1.1.6	Authentication Middleware	The system shall protect restricted routes using the isAuthenticatedUser Express middleware, which extracts and verifies the Bearer token from the Authorization header before granting access.

1.2 Instrument & Accessory (Item) Management

This module enables administrators to manage musical instruments, accessories, and parts available for purchase. It is implemented using Node.js Express routes, MySQL2 queries joining the item and stock tables, Multer for image handling, and jQuery DataTables on the frontend with built-in export buttons for PDF and Excel.

ID	Requirement	Description
FR1.2.1	Add Item	The system shall allow authenticated users to add new instruments or accessories by submitting description, brand, category, cost price, selling price, quantity, and an image file. The item record is inserted into the item table with references to the selected brand_id and category_id, and a corresponding stock record is created in the stock table within the same operation.
FR1.2.2	Display Items	The system shall retrieve and display all available instruments and accessories by querying the item, stock, brand, and category tables via INNER JOIN operations, returning the result as JSON to be rendered by the frontend as Bootstrap cards (customer view) or a DataTable (admin view).
FR1.2.3	View Single Item	The system shall allow retrieval of a single instrument or accessory's details including description, brand name, category name, cost price, sell price, stock quantity, and image path using the item's ID as a parameter.
FR1.2.4	Update Item	The system shall allow authenticated users to update an existing instrument or accessory's description, brand, category, cost price, sell price, and image, and simultaneously update its stock quantity in the stock table via a PUT request with multipart/form-data.
FR1.2.5	Delete Item	The system shall allow authenticated users to permanently delete an instrument or accessory record from the database via a DELETE request, with a Bootbox confirmation dialog on the frontend before proceeding.

ID	Requirement	Description
FR1.2.6	Item Image Upload	The system shall accept only .jpg, .jpeg, and .png image files for item uploads, validated by Multer's fileFilter, stored in the server-side images directory with a unique timestamped filename.

1.3 Shopping Cart Management

This module allows customers to temporarily store selected instruments and accessories before completing a purchase. Cart data is persisted client-side using localStorage via vanilla JavaScript, enabling cart state to survive page navigation without requiring a backend call.

ID	Requirement	Description
FR1.3.1	Add to Cart	The system shall allow customers to add instruments or accessories to their shopping cart from the product detail modal, specifying a quantity; if the item already exists in the cart, its quantity shall be incremented accordingly.
FR1.3.2	View Cart	The system shall display the current cart contents in a tabular format showing item image, description, unit price, quantity, subtotal per item, and a computed grand total.
FR1.3.3	Remove from Cart	The system shall allow customers to remove individual items from the cart by clicking a remove button, which splices the item from the localStorage cart array and re-renders the cart table.
FR1.3.4	Cart Persistence	The system shall persist the cart across page sessions by storing the cart array as a JSON string in localStorage, which is parsed and restored on each page load.
FR1.3.5	Cart Total Display	The system shall dynamically compute and display the cart's grand total price in Philippine Peso (PHP) by summing each item's unit price multiplied by its quantity.

1.4 Order and Checkout Management

This module processes customer purchases using MySQL transactions to ensure data integrity across the orderinfo and orderline tables. Upon successful checkout, the system sends an order confirmation email using Nodemailer configured via environment variables, with the customer's email retrieved through a JOIN between the users and customer tables.

ID	Requirement	Description
FR1.4.1	Place Order	The system shall allow authenticated customers to submit their cart as an order via a POST request; the

ID	Requirement	Description
		backend shall begin a MySQL transaction, insert a record into orderinfo, and insert one record per cart item into orderline, committing only when all insertions succeed.
FR1.4.2	Order Date Recording	The system shall automatically record the date_placed and date_shipped timestamps at the time of order creation.
FR1.4.3	Shipping Fee	The system shall apply a fixed shipping fee (currently set to 100) to each order at the time of orderinfo insertion.
FR1.4.4	Transaction Rollback	The system shall roll back the entire MySQL transaction if any error occurs during order insertion, preventing partial or corrupted order records.
FR1.4.5	Order Confirmation Email	The system shall send an HTML order confirmation email to the customer's registered email address upon successful order placement using Nodemailer with SMTP credentials stored in environment variables (.env).
FR1.4.6	Empty Cart Validation	The system shall reject checkout attempts with an empty cart and return an appropriate error response before any database operations are performed.
FR1.4.7	Post-Checkout Cart Clear	The system shall clear the localStorage cart on the client side upon a successful checkout response and re-render the empty cart view.

1.5 Customer Profile Management

This module manages the extended customer profile stored in the customer table, which is linked to the users table via a foreign key. It uses MySQL's INSERT ... ON DUPLICATE KEY UPDATE pattern to upsert customer data, and Multer for profile image handling.

ID	Requirement	Description
FR1.5.1	Upsert Customer Profile	The system shall create or update a customer's profile record (first name, last name, address line, zip code, phone, profile image) using INSERT ... ON DUPLICATE KEY UPDATE to handle both new and existing customer entries linked by user_id.
FR1.5.2	Profile Image Upload	The system shall allow customers to upload a profile image via a multipart/form-data POST request; the file

ID	Requirement	Description
		is processed by Multer, stored in the images directory, and the path is saved in the customer table.
FR1.5.3	Profile Image Preview	The system shall display a live preview of the selected profile image on the frontend using the FileReader API before the upload is submitted.

1.6 Dashboard and Reporting

This module provides authenticated administrators with visual analytics using Chart.js on the frontend. Data is aggregated via MySQL GROUP BY queries on the backend and served as JSON through protected Express routes. Three chart types are supported: bar, line, and pie.

ID	Requirement	Description
FR1.6.1	Customer Address Distribution Chart	The system shall generate a horizontal bar chart displaying the count of customers grouped by address line, using data aggregated from the customer table ordered by total count descending.
FR1.6.2	Monthly Sales Chart	The system shall generate a line chart displaying total monthly revenue by computing the sum of (quantity * sell_price) across orderline and item joined with orderinfo, grouped by month of date_placed.
FR1.6.3	Items Sold Distribution Chart	The system shall generate a pie chart displaying the total quantity sold per instrument or accessory, aggregated from the orderline table joined with the item table, grouped by item description.
FR1.6.4	Dashboard Authentication	The system shall restrict access to all dashboard chart data endpoints using the isAuthenticatedUser middleware, requiring a valid JWT Bearer token in the Authorization header.
FR1.6.5	DataTable Export	The system shall allow administrators to export the instrument and accessory list from the DataTable as PDF or Excel using the built-in DataTables Buttons extension.

1.7 Security and Access Control

This module defines how the system enforces authentication and protects sensitive routes. JWT-based stateless authentication is implemented server-side, while client-side session management uses sessionStorage to store tokens.

ID	Requirement	Description
FR1.7.1	JWT Authentication	The system shall use JSON Web Tokens (JWT) signed with a secret key stored in environment variables to authenticate users; the token is issued upon login and must be sent as a Bearer token in the Authorization header for all protected endpoints.
FR1.7.2	Password Hashing	The system shall hash all user passwords using bcrypt with a salt round of 10 before storing them in the database, and use bcrypt.compare for password verification during login.
FR1.7.3	Soft Delete / Account Deactivation	The system shall implement soft deletion for user accounts by setting a deleted_at timestamp rather than removing the record, and shall exclude soft-deleted users from login validation queries.
FR1.7.4	Unauthorized Access Redirect	The system shall detect the absence of a valid JWT token in sessionStorage on protected frontend pages and redirect unauthenticated users to the login page using SweetAlert2 (Swal) notification before redirection.
FR1.7.5	Environment Variable Configuration	The system shall store all sensitive configuration values including database credentials, JWT secret, and SMTP credentials in a .env file loaded via dotenv, and shall not hardcode credentials in source files.
FR1.7.6	CORS Support	The system shall enable Cross-Origin Resource Sharing (CORS) via the cors middleware to allow the vanilla JavaScript frontend to communicate with the Node.js backend running on a different port.

1.8 Email Notification

This module handles automated email notifications using Nodemailer. SMTP configuration is loaded from environment variables, supporting any compatible SMTP provider. Emails are sent as HTML messages.

ID	Requirement	Description
FR1.8.1	Order Confirmation Email	The system shall automatically send an HTML email notification to the customer's registered email address upon successful order placement, informing them that their order is being processed.
FR1.8.2	SMTP Configuration	The system shall configure the Nodemailer transporter using SMTP host, port, email, and password values defined in the .env file, supporting configurable mail providers.
FR1.8.3	Email Error Handling	The system shall handle email sending failures gracefully without interrupting the order completion response; email errors shall be logged to the server console but shall not cause the order transaction to fail.

1.9 Category Management

This module enables administrators to manage product categories used to classify musical instruments, accessories, and parts. Categories are stored in the category table and are referenced by item records through foreign key relationships.

ID	Requirement	Description
FR1.9.1	Add Item	The system shall allow authenticated administrators to create a new category by submitting a category name via an AJAX POST request to the backend; the category record is inserted into the category table.
FR1.9.2	Display Items	The system shall retrieve and display all categories from the category table in a DataTable with sorting, searching, and pagination features via AJAX calls to the backend.
FR1.9.3	View Single Item	The system shall allow administrators to retrieve and view the details of a specific category using its category_id as a parameter.
FR1.9.4	Update Item	The system shall allow authenticated administrators to modify an existing category's name through an AJAX PUT request, updating the category table record.

ID	Requirement	Description
FR1.9.5	Delete Item	The system shall allow authenticated administrators to delete a category through a DELETE request, provided that no items are currently associated with the category (foreign key constraint).
FR1.9.6	Item Image Upload	The system shall prevent the creation of duplicate category names by validating category records in the backend before insertion.

1.10 Brand Management

This module enables administrators to manage product brands associated with musical instruments, accessories, and parts. Brands are stored in the brand table and are referenced by item records through foreign key relationships.

ID	Requirement	Description
FR1.10.1	Add Brand	The system shall allow authenticated administrators to create a new brand by submitting a brand name via an AJAX POST request to the backend; the brand record is inserted into the brand table.
FR1.10.2	Display Brands	The system shall retrieve and display all brands from the brand table in a DataTable with sorting, searching, and pagination features via AJAX calls to the backend.
FR1.10.3	View Single Brands	The system shall allow administrators to retrieve and view the details of a specific brand using its brand_id as a parameter.
FR1.10.4	Update Brand	The system shall allow authenticated administrators to modify an existing brand's name through an AJAX PUT request, updating the brand table record.
FR1.10.5	Delete Brand	The system shall allow authenticated administrators to delete a brand through a DELETE request, provided that no items are currently associated with the brand (foreign key constraint).
FR1.10.6	Brand Validation	The system shall prevent the creation of duplicate brand names by validating brand records in the backend before insertion.